

北京交通大学软件综合实训

20 组_软件开发环境搭建说明

Development Environment Setup Guide

项目名称	北京交通大学就餐仿真系统
分组编号	20 组
文档版本	V1.0
编制说明	依据课程任务书、设计规格说明书、交付物要求明细和文档格式要求编制，重点说明第 20 组开发环境的安装、配置与验证流程。

北京交通大学计算机科学与技术学院

2026 年 03 月 27 日

目 录

- 1 编写目的与适用范围 1
 - 1.1 编制依据 1
 - 1.2 技术路线概览 1
- 2 开发环境与软件清单 2
 - 2.1 必装软件与版本 2
 - 2.2 前端与协作配套软件 2
- 3 开发工具安装与基础配置 3
 - 3.1 Python 3.11 安装 3
 - 3.2 IDE 与 Git 安装 4
 - 3.3 Node.js、前端工具链与浏览器准备 4
- 4 项目依赖安装与工程初始化 5
 - 4.1 推荐目录结构 5
 - 4.2 后端虚拟环境与依赖 5
 - 4.3 前端工程初始化 6
 - 4.4 配置文件与环境变量约定 6
- 5 HelloWorld 工程验证 7
 - 5.1 后端健康检查示例 7
 - 5.2 前后端联通验证示例 7
 - 5.3 环境搭建成功判定标准 8
- 6 常见问题与处理建议 8
- 7 小组统一约定 9

说明：目录页码按正文起始页重新编号，封面与目录不计入正文页码。

1 编写目的与适用范围

1.1 编制依据

根据课程任务书，立项阶段需要单独提交《[分组编号]_软件开发环境搭建说明》，内容至少覆盖开发工具下载安装、SDK/插件/依赖安装、环境变量配置以及 HelloWorld 工程验证流程。第 20 组在设计规格说明书中已确定项目采用“浏览器前端 + 本地后端仿真服务”的 B/S 路线，并明确前端使用 Vue 3 技术栈、后端使用 Python 3.11 + FastAPI 技术栈，因此本说明按既定实施方案编制。

本文档同时吸收课程简介中关于“小规模起步、保留扩展空间、重视过程记录”的要求，强调先完成可运行的参数配置—仿真执行—记录导出闭环，再逐步增加图表、推荐与 LLM 解释能力。文档既服务于组内统一环境，也作为后续开发、测试与集成阶段的可复用操作手册。

1.2 技术路线概览

结合规格说明书和小组已确定方案，第 20 组的开发路线为：浏览器端采用 Vue 3 + Element Plus + ECharts 实现参数配置、运行监控与结果展示；Python 后端采用 FastAPI + Uvicorn 提供仿真计算、过程记录、指标分析、优化推荐与解释生成服务；SQLite/CSV/JSON 用于保存配置快照、过程记录和导出数据。该路线已作为小组统一实施方案执行，不再保留前端技术栈分支选择。

- 前端统一采用 Node.js 20 LTS + Vite 工程构建方式，负责参数输入、状态刷新、结果展示与推荐说明等交互页面。
- 后端统一采用 Python 3.11 + venv + FastAPI + Uvicorn + Pydantic + pytest + pandas，优先完成参数校验、健康检查接口、仿真运行、结果汇总与导出。
- 增强说明模块：LLM 只作为解释增强层，不参与核心仿真计算；即使外部模型不可用，系统也必须能输出规则化说明。

2 开发环境与软件清单

2.1 必装软件与版本

必装软件以“保证本地能够完成编写、运行、调试、版本管理和测试”为准，不要求一次性安装全部高阶工具。对于不同成员的个人电脑，Windows 11 和 Ubuntu 22.04 均可使用，但组内统一约定 Python 版本和依赖版本，以减少联调差异。

表 2-1 第 20 组必装开发环境清单

类别	推荐版本/选型	是否必装	主要用途	验证方式
操作系统	Windows 11 或 Ubuntu 22.04	是	运行浏览器、Python、IDE 与 Git	系统可正常联网并打开终端
Python	3.11.x	是	实现核心仿真、数据分析、接口服务与测试	python --version
IDE	PyCharm 或 VS Code	是	代码编写、调试、断点跟踪	可成功创建并运行 Python 文件
Git	2.4x 及以上	是	版本管理、分支协作、提交留痕	git --version
浏览器	Chrome / Edge 最新稳定版	是	访问本地前端页面和 API 文档	可访问 http://127.0.0.1:8000/docs
pytest	8.x	是	单元测试与回归检查	pytest --version

2.2 前端与协作配套软件

以下软件分为“统一安装的前端工具链”和“按角色建议安装的辅助工具”两类。其中 Node.js、Vue 3、Element Plus、ECharts 和 axios 属于第 20 组既定技术路线的统一环境；原型绘图和数据库查看工具用于提升协作效率。

表 2-2 前端与辅助工具清单

工具	推荐版本/选型	安装要求	主要用途
Node.js	20 LTS	统一安装	使用 Vue 3/Vite 构建前端工程
Vue 3 + Vite	最新版稳定版	统一安装	实现参数配置页、运行页与结果展示页
Element Plus	最新版稳定版	统一安装	快速构建表单、表格、按钮和布局组件
ECharts	5.x	统一安装	展示队列长度、空座位和吞吐变化曲线
draw.io / ProcessOn	在线版即可	建议安装（原型/文档负责人）	绘制模块图、数据流图和原型草图
SQLite 可视化工具	DB Browser for SQLite	建议安装（数据/测试负责人）	查看过程记录与汇总结果

3 开发工具安装与基础配置

3.1 Python 3.11 安装

Python 3.11 是第 20 组的统一后端运行环境。Windows 用户建议从官方安装包安装，并在安装界面勾选“Add python.exe to PATH”；Ubuntu 用户可使用系统包管理器或 pyenv 安装，但最终需要确保命令行中 python 和 pip 指向 3.11 环境。

- Windows: 安装完成后在 PowerShell 或 CMD 中执行 `python --version` 与 `pip --version`，确认命令可直接识别。
- Ubuntu: 可使用 `sudo apt install python3.11 python3.11-venv python3-pip`，随后通过 `python3.11 --version` 验证。
- 建议统一升级 pip: `python -m pip install --upgrade pip`；后续依赖一律在虚拟环境中安装，不直接污染系统环境。

```
Windows PowerShell
python --version
pip --version
```

```
python -m pip install --upgrade pip
```

Ubuntu

```
python3.11 --version
```

```
python3.11 -m pip install --upgrade pip
```

3.2 IDE 与 Git 安装

IDE 推荐 PyCharm 或 VS Code，二者任选其一即可。若使用 VS Code，建议额外安装 Python、Pylance、Jupyter 以及 Vue Language Features（前端成员）。Git 建议所有成员安装并完成全局身份配置，确保后续提交记录能够直接对应到个人。

- **PyCharm:** 新建项目时直接选择已有虚拟环境，便于统一解释器路径。
- **VS Code:** 打开项目根目录后，通过命令面板选择正确的 Python 解释器。
- **Git:** 安装完成后执行 `git config --global user.name` 与 `git config --global user.email`，提交前先确认身份信息无误。

```
git --version
```

```
git config --global user.name "第 20 组成员姓名"
```

```
git config --global user.email "your_email@example.com"
```

```
# Windows 建议: git config --global core.autocrlf true
```

3.3 Node.js、前端工具链与浏览器准备

第 20 组前端已确定采用 Vue 3 + Vite 路线，因此前端相关成员需安装 Node.js 20 LTS。安装完成后执行 `node -v` 与 `npm -v` 验证。浏览器方面，推荐 Chrome 或 Edge 最新稳定版，用于访问本地前端页面和 FastAPI 自动生成的 API 文档。

```
node -v
```

```
npm -v
```

```
npm config get registry
```

```
# 若网络较慢，可在课程允许范围内配置更快的镜像源
```

4 项目依赖安装与工程初始化

4.1 推荐目录结构

为兼顾立项阶段可读性与后续迭代扩展，建议采用“文档、后端、前端、测试、数据、脚本”分层目录。这样既便于功能拆分，也利于后续提交源代码说明文档。

DiningSimulation20/
├─ backend/ # FastAPI、仿真逻辑、分析与导出
├─ frontend/ # Vue 3 工程与页面资源
├─ tests/ # pytest 测试用例
├─ data/ # SQLite、CSV、JSON 与示例输入
├─ docs/ # 规格说明书、沟通记录、原型图
├─ scripts/ # 启动脚本、初始化脚本
└─ README.md # 项目说明与运行方法

4.2 后端虚拟环境与依赖

后端初始化采用 venv 虚拟环境。第 20 组当前版本统一安装 FastAPI、Uvicorn、Pydantic、pytest 和 pandas；SQLite 以及 json、csv 等标准库无需单独安装。

```
cd DiningSimulation20/backend
python -m venv .venv
# Windows 激活
.venv\Scripts\activate
# Linux/macOS 激活
source .venv/bin/activate
python -m pip install --upgrade pip
pip install fastapi uvicorn pydantic pytest pandas
pip freeze > requirements.txt
```

表 4-1 后端推荐依赖清单

依赖	是否纳入统一依赖	用途说明
fastapi	是	提供配置校验、单步运行、指标查询等 REST 接口
uvicorn	是	启动本地开发服务器
pydantic	是	进行配置项和 DTO 的结构化校验
pytest	是	编写并执行单元测试
pandas	是	用于过程记录汇总、指标统计与结果导出

4.3 前端工程初始化

由于前端路线已确定为 Vue 3，项目统一使用 Vite 创建 frontend 工程，并安装 Element Plus、ECharts 与 axios。立项阶段至少完成参数输入页、健康检查联通页和结果展示占位页三个基础页面，为后续仿真接入预留接口。

```
cd DiningSimulation20
npm create vite@latest frontend -- --template vue
cd frontend
npm install
npm install element-plus echarts axios
npm run dev
```

4.4 配置文件与环境变量约定

系统级必须配置包括 PATH 中能够找到 Python、pip、Git、Node 和 npm。除此之外，第 20 组不建议在操作系统层面写入过多机器相关变量，而是通过项目根目录下的 .env 或 config.json 保存运行配置，降低换机和联调成本。

表 4-2 推荐项目级配置项

配置项	示例值	作用说明
APP_ENV	dev	标识当前为开发环境
HOST	127.0.0.1	后端本地监听地址
PORT	8000	后端服务端口
DATA_DIR	./data	过程记录、导出文件存放目录
LLM_ENABLED	false	立项阶段默认关闭，避免依赖外部服务

- Windows PowerShell 如提示脚本被禁止执行，可改用 CMD 激活虚拟环境，或在了解风险前提下为当前用户放宽本地脚本执行策略。
- 项目路径尽量避免过长或包含非常规符号，减少前端构建、SQLite 路径和脚本执行中的兼容性问题。

5 HelloWorld 工程验证

5.1 后端健康检查示例

HelloWorld 验证的目标不是展示完整功能，而是确认“解释器、依赖、服务启动、浏览器访问和接口返回”五个环节都正常。第 20 组建议先实现一个最小健康检查接口，再在此基础上接入仿真逻辑。

```
from fastapi import FastAPI

app = FastAPI(title="Dining Simulation Group 20")

@app.get("/api/health")
def health():
    return {"group": "20 组", "status": "ok", "message": "backend ready"}

cd DiningSimulation20/backend
.venv\Scripts\activate # Linux/macOS 请改为 source .venv/bin/activate
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
# 浏览器访问 http://127.0.0.1:8000/api/health 或 /docs
```

5.2 前后端联通验证示例

前端验证统一使用 Vue 页面通过 axios 调用 /api/health。只要浏览器页面能正确显示“20 组 backend ready”或等价内容，即可判定前后端最小联通成功。

```
<!DOCTYPE html>
<html lang="zh-CN">
<body>
  <button onclick="loadHealth()">测试环境</button>
  <pre id="result"></pre>
  <script>
    async function loadHealth(){
      const res = await fetch("http://127.0.0.1:8000/api/health");
      document.getElementById("result").textContent = JSON.stringify(await
res.json(), null, 2);
    }
  </script>
</body>
</html>
```

5.3 环境搭建成功判定标准

- 命令行能够正确输出 Python、pip、Git、node 和 npm 版本号。

- 后端虚拟环境可正常激活，requirements.txt 中的依赖能够成功安装，无持续性的红色报错。
- uvicorn 启动后浏览器可访问 /api/health 和 /docs，返回内容与示例一致。
- 代码能够在 IDE 中设置断点、单步执行，并在仓库中完成首次提交。
- HelloWorld 运行截图、命令记录和问题处理过程已保留到 docs/ 或小组共享盘，便于过程考核与后续报告编写。

6 常见问题与处理建议

立项阶段最常见的问题不是业务逻辑本身，而是环境差异导致的“装好了但跑不起来”。为降低重复沟通成本，第 20 组将常见问题、现象和处理建议整理如下，后续出现新问题时继续补充。

表 6-1 环境搭建常见问题与处理建议

现象	可能原因	处理建议
python 命令无法识别	未加入 PATH 或终端未重启	重新安装并勾选加入 PATH；关闭后重新打开终端再验证
虚拟环境无法激活	PowerShell 执行策略限制或路径错误	确认激活脚本路径；必要时改用 CMD 或为当前用户调整策略
8000 端口被占用	已有服务正在运行	关闭占用进程，或在启动 uvicorn 时改用其他端口
npm install 很慢	网络状况不稳定	优先在网络较好的时段安装，必要时使用课程允许的镜像源
接口能访问但前端 fetch 失败	地址、端口或跨域配置不一致	核对请求 URL；同源调试优先；必要时临时开放开发环境跨域
SQLite 文件写入失败	data 目录不存在或权限不足	先创建 data 目录，并避免将数据库放入只读目录

7 小组统一约定

为了让环境说明真正服务于后续开发，而不是只在立项阶段提交一次后搁置，第 20 组对开发环境和协作方式形成如下统一约定：

- 所有成员统一使用 Python 3.11，避免 3.10/3.12 混用造成依赖差异。
- 后端依赖以 requirements.txt 为准；需要新增依赖时，必须在沟通后同步更新文件并说明用途。
- 主仓库按模块建立分支，例如 feature/sim-core、feature/analysis、feature/vis；禁止在未沟通情况下直接覆盖他人代码。
- 立项、开发、集成和部署阶段均保留沟通记录、运行截图、测试记录和 AI 使用提示词，作为过程考核证据。
- 阶段文档最终统一导出为 PDF 提交，命名遵循课程命名规范；源代码和阶段文档按课程要求分别打包归档。

通过以上安装、配置和验证流程，第 20 组可以在不同成员电脑上构建一致的本地开发环境，为后续核心仿真模块、分析优化模块和可视化/LLM 模块并行开发提供稳定基础。